國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 07

姓名：簡紘田

學號：114310115

Lab07-Q1.(題目編號)

Q1: Complete the following parts in the sample code:

- precedence

- InfixToPostfix

Q2: Add comments to the program to clearly describe its actions.

Q3: Draw a table showing the step-by-step process of converting each infix expression to postfix (following the method presented in the lecture notes.).

Code

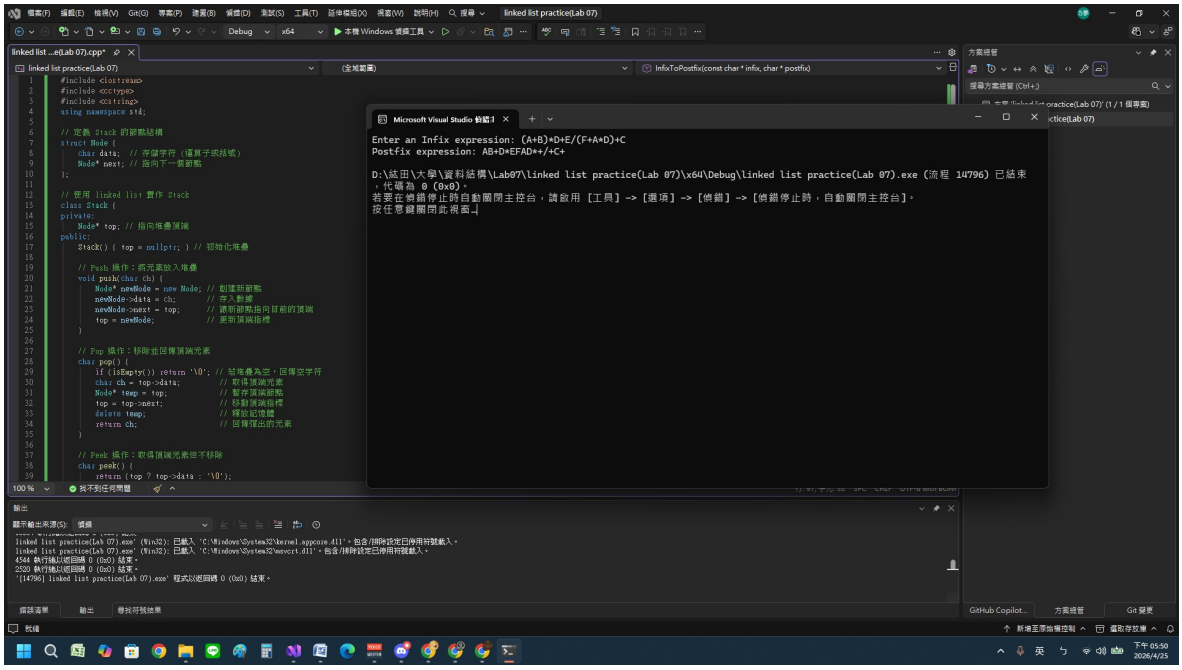
```
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

// 定義 Stack 的節點結構
struct Node {
    char data;  // 存儲字符 (運算子或括號)
    Node* next; // 指向下一個節點
};

// 使用 linked list 實作 Stack
class Stack {
private:
    Node* top; // 指向堆疊頂端
public:
    Stack() { top = nullptr; } // 初始化堆疊

    // Push 操作：將元素放入堆疊
    void push(char ch) {
        Node* newNode = new Node; // 創建新節點
        newNode->data = ch;       // 存入數據
        newNode->next = top;      // 讓新節點指向目前的頂端
        top = newNode;            // 更新頂端指標
    }

    // Pop 操作：移除並回傳頂端元素
    char pop() {
        if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
```

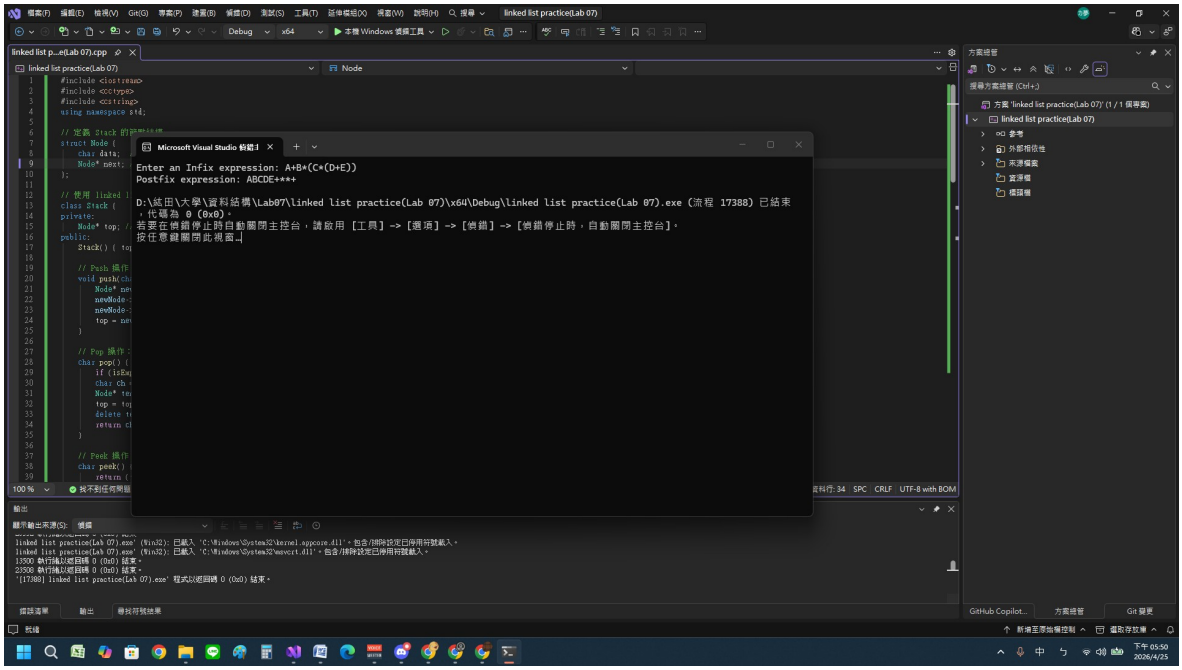
```
        char ch = top->data;          // 取得頂端元素
        Node* temp = top;             // 暫存頂端節點
        top = top->next;              // 移動頂端指標
        delete temp;                  // 釋放記憶體
        return ch;                    // 回傳彈出的元素
    }

    // Peek 操作：取得頂端元素但不移除
    char peek() {
        return (top ? top->data : '\0');
    }

    // 判斷堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }
};

// 判斷運算子(加減乘除) 的優先順序
int precedence(char op) {
    switch (op) {
    case '+':
    case '-': return 1; // 低優先權
    case '*':
    case '/': return 2; // 高優先權

    }
    return 0;
}

// 將中序表達式 (infix) 轉換為後序表達式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
    Stack s;
    int i = 0, j = 0;
    char token;

    while ((token = infix[i++]) != '\0') {  // 重複直到換行 (Enter)
```

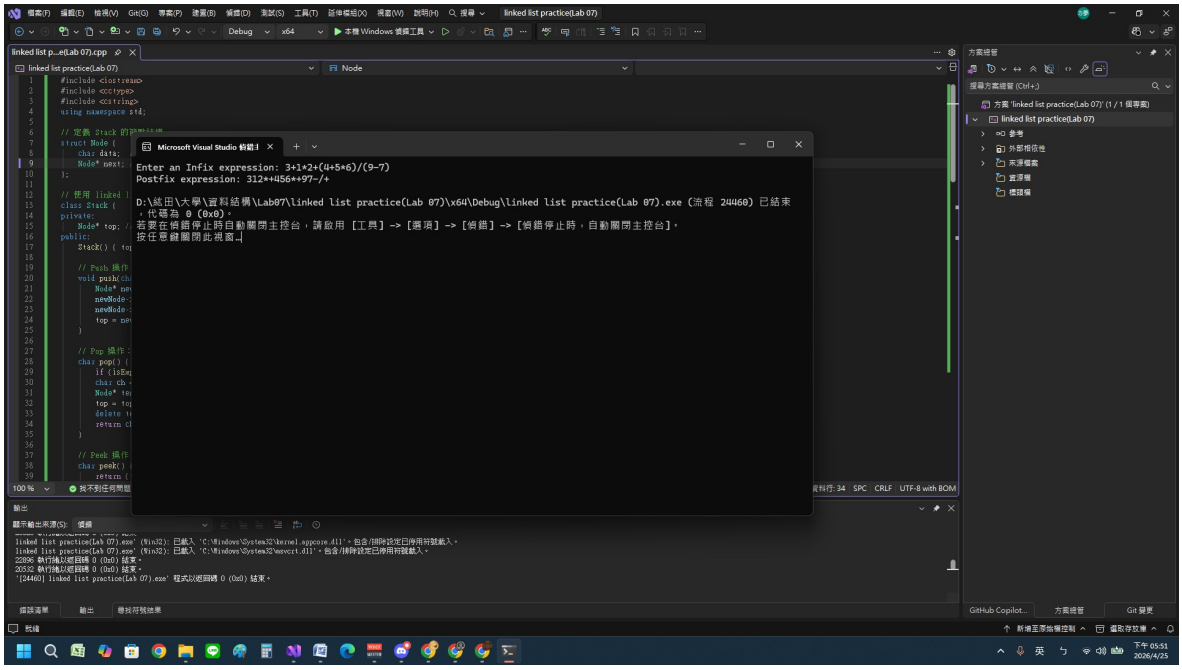
```
        // 將運算元直接輸出
        if (isalnum(token)) {
            postfix[j++] = token;
        }

        // 將左括號放進儲存
        else if (token == '(') {
            s.push(token);
        }

        // 將右括號的東西一直丟到左括號
        else if (token == ')') {
            while (!s.isEmpty() && s.peek() != '(') {
                postfix[j++] = s.pop();
            }
            s.pop(); // 丟掉左括號
        }

        // 運算子
        else {
            while (!s.isEmpty() && precedence(s.peek()) >= precedence(token)) {
                postfix[j++] = s.pop();
            }
            s.push(token);
        }
    }

    // 清空堆疊
    while (!s.isEmpty()) {
        postfix[j++] = s.pop();
    }

    postfix[j] = '\0';
}

int main() {
    char infix[100], postfix[100];
    cout << "Enter an Infix expression: ";
```

```
    cin >> infix; // 輸入中序表達式

    InfixToPostfix(infix, postfix); // 轉換為後序表達式
    cout << "Postfix expression: " << postfix << endl; // 輸出後序表達式

    return 0;
}
```

Result




Microsoft Visual Studio 偵錯主
Enter an Infix expression: A+B*(C*(D+E))
Postfix expression: ABCDE+**+
D:\紙田\大學\資料結構\Lab07\linked list practice(Lab 07)\x64\Debug\linked list practice(Lab 07).exe (流程 17388) 已結束，代碼為 0 (0x0)。
若要在偵錯停止時自動關閉主控台，請啟用 [工具] -> [選項] -> [偵錯] -> [偵錯停止時，自動關閉主控台]。
按任意鍵關閉此視窗...


Microsoft Visual Studio 偵錯主
Enter an Infix expression: 3+1*2+(4+5*6)/(9-7)
Postfix expression: 312*+456*+97-/+
D:\紙田\大學\資料結構\Lab07\linked list practice(Lab 07)\x64\Debug\linked list practice(Lab 07).exe (流程 24460) 已結束，代碼為 0 (0x0)。
若要在偵錯停止時自動關閉主控台，請啟用 [工具] -> [選項] -> [偵錯] -> [偵錯停止時，自動關閉主控台]。
按任意鍵關閉此視窗...

Discussion

(A+B)*D+E/(F+A*D)+C:

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
(	(					0	
A	(					0	A
+	(	+				1	A
B	(	+				1	AB
)						-1	AB+
*	*					0	AB+
D	*					0	AB+D
+	+					0	AB+D*
E	+					0	AB+D*E
/	+	/				1	AB+D*E
(	+	/	(			2	AB+D*E
F	+	/	(			2	AB+D*EF
+	+	/	(	+		3	AB+D*EF
A	+	/	(	+		3	AB+D*EFA
*	+	/	(	+	*	4	AB+D*EFA
D	+	/	(	+	*	4	AB+D*EFAD
)	+	/				1	AB+D*EFAD+/
+	+					0	AB+D*EFAD+/+
C	+					0	AB+D*EFAD+/+C
<eos>						-1	AB+D*EFAD+/+C+

A+B*(C*(D+E))

Token	Stack						Top	Output
	[0]	[1]	[2]	[3]	[4]	[5]		
A							-1	A
+	+						0	A
B	+						0	AB
*	+	*					1	AB
(	+	*	(				2	AB
C	+	*	(				2	ABC
*	+	*	(	*			3	ABC
(	+	*	(	*	(		4	ABC
D	+	*	(	*	(		4	ABCD
+	+	*	(	*	(	+	5	ABCD

E	+	*	(	*	(	+	5	ABCDE
)	+	*	(	*			2	ABCDE+
)	+	*					1	ABCDE+*
<eos>							-1	ABCDE+**+

3+1*2+(4+5*6)/(9-7)

Token	Stack					Top	Output
	[0]	[1]	[2]	[3]	[4]		
3						-1	3
+	+					0	3
1	+					0	31
*	+	*				1	31
2	+	*				1	312
+	+					0	312
(	+	(				1	312*+
4	+	(				1	312*+
+	+	(	+			2	312*+4
5	+	(	+			2	312*+4
*	+	(	+	*		3	312*+45
6	+	(	+	*		3	312*+45
)	+					0	312*+456
/	+	/				1	312*+456*+
(	+	/	(			2	312*+456*+
9	+	/	(			2	312*+456*+9
-	+	/	(			3	312*+456*+9
7	+	/	(			3	312*+456*+97
)	+	/				1	312*+456*+97-
<eos>						-1	312*+456*+97-/+